

Revisão para a Prova — Análise de Algoritmos

Prova com consulta (off-line) — terça-feira, 22/09/2026 · Fatec Jundiaí · Prof. Humberto Zanetti

PARTE 1 — Resumo para consulta rápida

- 1.1 Notação Big O em 6 frases
- 1.2 As classes de complexidade
- 1.3 A tabela do livro: computador lento de 10 operações por segundo, $n = 16$
- 1.4 Melhor, médio e pior caso dos algoritmos vistos
- 1.5 Fórmulas que caem em conta
- 1.6 Dividir para conquistar (DC) — a receita
- 1.7 Como medir um algoritmo sem se enganar (aprendido nas atividades 01, 02 e 03)

PARTE 2 — Códigos de referência (Python)

- 2.1 Busca linear e busca binária
- 2.2 Selection sort (livro, cap. 2) e bubble sort (atividade 02)
- 2.3 Quicksort (aula 06 / livro, cap. 4) e merge sort
- 2.4 Recursão: soma, contagem, máximo, fatorial, Euclides
- 2.5 Fibonacci: recursivo puro, memoizado e iterativo (atividade 03)
- 2.6 Medindo tempo

PARTE 3 — Exercícios com respostas

- Tema 1 — Notação Big O e crescimento
- Tema 2 — Busca linear × busca binária
- Tema 3 — Ordenação: selection sort e bubble sort
- Tema 4 — Recursividade
- Tema 5 — Fibonacci: recursivo puro × memorização (cache)
- Tema 6 — Dividir para conquistar e Quicksort
- Tema 7 — Questões discursivas e cenários (estilo prova)

PARTE 4 — Simulado rápido (gabarito no fim da parte)

PARTE 5 — Tabelas numéricas para consulta

- 5.1 Potências de 2 (passos da busca binária = k para $n = 2^k$)
- 5.2 $n(n-1)/2$ — comparações do selection sort e do bubble sort (pior caso)
- 5.3 Fibonacci e número de chamadas
- 5.4 Fatoriais (rotas do caixeiro-viajante)
- 5.5 Logaritmos úteis (base 2)

Base deste material: aulas 03 a 06 (Big O, busca linear × binária, bubble × selection sort, recursividade e Fibonacci com cache, dividir para conquistar e quicksort) e os capítulos 1 a 4 do livro *Entendendo Algoritmos* (Aditya Y. Bhargava). Os exercícios marcados com **(Livro, ex. X.Y)** são os do livro, com a resposta do próprio autor comentada.

Como usar: a Parte 1 é o resumo para consulta rápida; a Parte 2 traz os códigos prontos; a Parte 3 tem os exercícios com a resposta logo abaixo de cada um; a Parte 4 é um simulado com gabarito no fim; a Parte 5 tem tabelas numéricas (potências de 2, Fibonacci, $n(n-1)/2$) para não perder tempo com conta na prova.

Material complementar: o **Catálogo de Scripts** (Catalogo_Scripts.pdf e a pasta scripts/) traz 20 scripts executáveis, cada um com o código comentado, a saída real e as perguntas prováveis sobre ele, com índice por assunto, por pergunta e por palavra-chave.

PARTE 1 — Resumo para consulta rápida

1.1 Notação Big O em 6 frases

1. **Big O** descreve como o número de operações cresce quando o tamanho da entrada (n) cresce. É uma taxa de crescimento, não um tempo em segundos.
2. Por padrão, **Big O** fala do pior caso: “no máximo este tanto de operações”.
3. Ignora constantes e termos menores: $3n^2 + 5n + 10$ é $O(n^2)$; $n/26$ é $O(n)$; $20 \cdot \log n + 4$ é $O(\log n)$.
4. Contar passos é propriedade do algoritmo (igual em qualquer máquina). Medir tempo é propriedade da execução (depende de CPU, linguagem, cache, sistema operacional).
5. Um algoritmo $O(\log n)$ não vence um $O(n)$ em todo caso particular. A busca linear acha o 1º elemento em 1 passo; a binária precisa de vários. Big O fala do crescimento, não de cada instância.
6. Para n pequeno, as constantes mandam; para n grande, a classe Big O manda. Por isso quicksort (constante menor) costuma vencer merge sort na prática, mesmo com pior caso $O(n^2)$.

1.2 As classes de complexidade

Big O	Nome	Exemplos vistos no curso	Se n dobra, o trabalho...
$O(1)$	constante	acessar <code>lista[i]</code> ; consultar um dict (cache); dobrar só o 1º elemento	não muda
$O(\log n)$	logarítmico	busca binária; algoritmo de Euclides (maior quadrado da fazenda)	aumenta 1 passo
$O(n)$	linear	busca linear; percorrer uma lista; soma recursiva; Fibonacci memoizado ou iterativo	dobra ($\times 2$)
$O(n \log n)$	linearítmico	quicksort (caso médio); merge sort	um pouco mais que dobra
$O(n^2)$	quadrático	bubble sort; selection sort; quicksort (pior caso); tabela de multiplicação	quadruplica ($\times 4$)
$O(2^n)$	exponencial	Fibonacci recursivo puro ($\approx 1,618^n$); força bruta sobre subconjuntos	eleva ao quadrado (cada +1 em n quase dobra)
$O(n!)$	fatorial	caixeiro-viajante (testar todas as rotas)	explode

Ordem de crescimento (do mais rápido ao mais lento): $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

1.3 A tabela do livro: computador lento de 10 operações por segundo, $n = 16$

Big O	Operações ($n = 16$)	Tempo
$O(\log n)$	4	0,4 s
$O(n)$	16	1,6 s
$O(n \log n)$	64	6,4 s
$O(n^2)$	256	25,6 s

Big O	Operações (n = 16)	Tempo
O(n!)	16! $\approx 2,09 \times 10^{13}$	≈ 66 mil anos

1.4 Melhor, médio e pior caso dos algoritmos vistos

Algoritmo	Melhor	Médio	Pior	Memória extra	Observações
Busca linear	O(1)	O(n)	O(n)	O(1)	não exige ordenação; melhor caso = alvo na 1ª posição
Busca binária	O(1)	O(log n)	O(log n)	O(1)	exige lista ordenada ; passos $\approx \log_2 n$
Selection sort	O(n²)	O(n²)	O(n²)	O(1)	sempre $n(n-1)/2$ comparações; no máximo n-1 trocas
Bubble sort (com parada)	O(n)	O(n²)	O(n²)	O(1)	melhor caso: n-1 comparações, 0 trocas, 1 passagem; pior: $n(n-1)/2$ comparações e trocas
Quicksort	O(n log n)	O(n log n)	O(n²)	pilha O(log n) melhor, O(n) pior	pior caso: lista já ordenada + pivô fixo (1º elemento)
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	sempre n log n, mas constante maior que a do quicksort
Fibonacci recursivo puro	—	O(2 ⁿ) — exatamente $\Theta(\phi^n)$, $\phi \approx 1,618$	—	pilha O(n)	chamadas = $2 \cdot F(n+1) - 1$
Fibonacci memoizado (top-down, cache)	—	O(n)	—	cache O(n) + pilha O(n)	chamadas = $2n - 1$
Fibonacci iterativo (bottom-up)	—	O(n)	—	O(1)	sem risco de RecursionError
Soma / contagem / máximo recursivos	—	O(n) chamadas	—	pilha O(n)	
Euclides (maior quadrado)	—	O(log(min(a, b)))	—	pilha O(log)	

1.5 Fórmulas que caem em conta

- **Passos da busca binária (pior caso):** $\lceil \log_2 n \rceil$ — é como o livro conta. Uma implementação que conta cada comparação faz no máximo $\lceil \log_2 n \rceil + 1$. Exemplos: 128 $\rightarrow$ 7; 256 $\rightarrow$ 8; 240.000 $\rightarrow$ 18; 1.000.000 $\rightarrow$ 20; 4 bilhões $\rightarrow$ 32.

- **Comparações de selection sort (sempre) e bubble sort (pior caso):** $n(n-1)/2 = (n-1) + (n-2) + \dots + 1$. Para $n = 5$: 10; $n = 10$: 45; $n = 100$: 4.950; $n = 500$: 124.750.
- **Trocas:** bubble sort no pior caso = $n(n-1)/2$ (toda comparação troca); selection sort $\leq n-1$.
- **Chamadas do Fibonacci recursivo puro:** $T(n) = T(n-1) + T(n-2) + 1$, com solução $T(n) = 2 \cdot F(n+1) - 1$. Ex.: $n = 5 \rightarrow 15$; $n = 10 \rightarrow 177$; $n = 20 \rightarrow 21.891$; $n = 30 \rightarrow 2.692.537$.
- **Chamadas do Fibonacci memoizado:** $2n - 1$. Ex.: $n = 10 \rightarrow 19$; $n = 20 \rightarrow 39$; $n = 30 \rightarrow 59$.
- **Quicksort:** custo total = (custo por nível, $O(n)$) $\times$ (altura da pilha). Altura $\log n \rightarrow O(n \log n)$; altura $n \rightarrow O(n^2)$.
- **Quando n dobra:** $O(\log n) + 1$ passo $\cdot O(n) \times 2 \cdot O(n \log n) \times (2 + \text{um pouco}) \cdot O(n^2) \times 4 \cdot O(n^3) \times 8 \cdot O(2^n)$ vira $(2^n)^2$.
- **Quando n multiplica por 10:** $O(n) \times 10 \cdot O(n^2) \times 100 \cdot O(n \log n) \times (\approx 13 \text{ para } n \text{ na casa dos milhares})$.

1.6 Dividir para conquistar (DC) — a receita

1. **Descubra o caso base** — o caso mais simples possível (lista vazia ou com 1 elemento; um lado do terreno múltiplo do outro).
2. **Divida ou diminua o problema** até que ele vire o caso base.

Etapas: **Dividir** (quebrar em subproblemas menores) $\rightarrow$ **Conquistar** (resolver cada um recursivamente) $\rightarrow$ **Combinar** (juntar as soluções).

Sem caso base, a função nunca para: a pilha de chamadas cresce até estourar (**stack overflow**; em Python, `RecursionError: maximum recursion depth exceeded`).

1.7 Como medir um algoritmo sem se enganar (aprendido nas atividades 01, 02 e 03)

- **Medir passos E tempo.** Quando as duas evidências apontam na mesma direção, a conclusão não é artefato da máquina de teste.
 - **Validar antes de comparar.** Um algoritmo rápido e errado não serve como base de comparação — por isso o teste de correteza vem antes do benchmark.
 - **Repetir e usar mediana (ou mínimo), não média.** Uma medição isolada de 0,002 ms está dentro do ruído do relógio; média é puxada por picos do sistema operacional.
 - **Fixar a semente aleatória** (`random.seed(42)`) para que cada execução meça o mesmo teste.
 - **Zerar o cache entre medições** do algoritmo memoizado; senão você mede um acerto pronto, não o trabalho real.
 - **Usar lista grande.** Com n pequeno, a diferença entre $O(n)$ e $O(\log n)$ se perde nas constantes.
-

PARTE 2 — Códigos de referência (Python)

2.1 Busca linear e busca binária

```
def busca_linear(lista, alvo):
    passos = 0
    for i in range(len(lista)):          # olha um por um: O(n)
        passos += 1
        if lista[i] == alvo:
            return i, passos
    return None, passos                  # pior caso: percorreu tudo

def busca_binaria(lista, alvo):          # a lista PRECISA estar ordenada
    baixo, alto = 0, len(lista) - 1
    passos = 0
    while baixo <= alto:
        passos += 1
        meio = (baixo + alto) // 2
        chute = lista[meio]
        if chute == alvo:
            return meio, passos
        if chute < alvo:
            baixo = meio + 1             # descarta a metade da esquerda
        else:
            alto = meio - 1              # descarta a metade da direita
    return None, passos                  # O(log n)

def busca_binaria_rec(lista, alvo, baixo=0, alto=None): # versão "dividir para conquistar"
    if alto is None:
        alto = len(lista) - 1
    if baixo > alto:                     # caso base 1: acabou o intervalo -> não está
        return None
    meio = (baixo + alto) // 2
    if lista[meio] == alvo:              # caso base 2: achou
        return meio
    if lista[meio] < alvo:                # caso recursivo: metade da direita
        return busca_binaria_rec(lista, alvo, meio + 1, alto)
    return busca_binaria_rec(lista, alvo, baixo, meio - 1) # metade da esquerda
```

2.2 Selection sort (livro, cap. 2) e bubble sort (atividade 02)

```
def selection_sort(lista):
    lista = list(lista)                 # cópia: não altera a original
    n = len(lista)
    comparacoes = trocas = 0
    for i in range(n - 1):
        menor = i
        for j in range(i + 1, n):       # procura o menor no trecho restante
            comparacoes += 1
            if lista[j] < lista[menor]:
                menor = j
        if menor != i:
            lista[i], lista[menor] = lista[menor], lista[i]
            trocas += 1
    return lista, comparacoes, trocas   # SEMPRE n(n-1)/2 comparações

def bubble_sort(lista):
```

```

lista = list(lista)
n = len(lista)
comparacoes = trocas = 0
for passagem in range(n - 1):          # no máximo n-1 passagens
    houve_troca = False
    for i in range(n - 1 - passagem):  # a cauda já está ordenada
        comparacoes += 1
        if lista[i] > lista[i + 1]:
            lista[i], lista[i + 1] = lista[i + 1], lista[i]
            trocas += 1
            houve_troca = True
    if not houve_troca:                # passagem sem troca -> já ordenada, para (melhor caso O(n))
        break
return lista, comparacoes, trocas

```

2.3 Quicksort (aula 06 / livro, cap. 4) e merge sort

```

def quicksort(array):
    if len(array) < 2:                # caso base: 0 ou 1 elemento já está ordenado
        return array
    pivo = array[0]                   # pivô = primeiro elemento
    menores = [x for x in array[1:] if x <= pivo]
    maiores = [x for x in array[1:] if x > pivo]
    return quicksort(menores) + [pivo] + quicksort(maiores)

def quicksort_meio(array):
    if len(array) < 2:                # pivô do meio: evita o pior caso em lista ordenada
        return array
    m = len(array) // 2
    pivo = array[m]
    menores = [x for i, x in enumerate(array) if x < pivo and i != m]
    maiores = [x for i, x in enumerate(array) if x >= pivo and i != m]
    return quicksort_meio(menores) + [pivo] + quicksort_meio(maiores)

def merge_sort(array):
    if len(array) < 2:
        return array
    meio = len(array) // 2
    esquerda = merge_sort(array[:meio]) # dividir
    direita = merge_sort(array[meio:])
    saida, i, j = [], 0, 0
    while i < len(esquerda) and j < len(direita): # combinar (merge): O(n) por nível
        if esquerda[i] <= direita[j]:
            saida.append(esquerda[i]); i += 1
        else:
            saida.append(direita[j]); j += 1
    return saida + esquerda[i:] + direita[j:]

```

2.4 Recursão: soma, contagem, máximo, fatorial, Euclides

```

def soma(lista):
    if lista == []:                  # aula 06 / Livro ex. 4.1
        return 0                     # caso base
    return lista[0] + soma(lista[1:]) # caso recursivo

def contar(lista):
    if lista == []:                  # Livro ex. 4.2
        return 0

```

```

    return 1 + contar(lista[1:])

def maximo(lista):
    # Livro ex. 4.3
    # caso base: 1 elemento é o próprio máximo
    if len(lista) == 1:
        return lista[0]
    resto = maximo(lista[1:])
    return lista[0] if lista[0] > resto else resto

def fatorial(n):
    # Livro cap. 3
    if n == 1:
        return 1
    return n * fatorial(n - 1)

def maior_quadrado(lado1, lado2):
    # aula 06: fazenda 1680 x 640 -> 80 (algoritmo de Euclides / MDC)
    if lado1 == 0 or lado2 == 0:
        return 0
    elif lado1 % lado2 == 0:
        # caso base: um lado é múltiplo do outro
        return lado2
    else:
        return maior_quadrado(lado2, lado1 % lado2) # reduz o problema

```

2.5 Fibonacci: recursivo puro, memoizado e iterativo (atividade 03)

```

from functools import lru_cache

def fib_recursivo(n):
    #  $O(2^n)$  tempo,  $O(n)$  pilha
    #  $F(0) = 0$ ,  $F(1) = 1$ 
    if n < 2:
        return n
    return fib_recursivo(n - 1) + fib_recursivo(n - 2)

def fib_memo(n, cache=None):
    #  $O(n)$  tempo,  $O(n)$  memória (top-down)
    if cache is None:
        cache = {0: 0, 1: 1}
    if n in cache:
        # já calculado -> consulta  $O(1)$ 
        return cache[n]
    cache[n] = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
    return cache[n]

@lru_cache(maxsize=None)
def fib_lru(n):
    # a mesma ideia, com o cache da biblioteca padrão
    if n < 2:
        return n
    return fib_lru(n - 1) + fib_lru(n - 2)

def fib_iterativo(n):
    #  $O(n)$  tempo,  $O(1)$  memória (bottom-up)
    anterior, atual = 0, 1
    for _ in range(n):
        anterior, atual = atual, anterior + atual
    return anterior

```

2.6 Medindo tempo

```

import time, timeit, random, statistics

```

```
inicio = time.perf_counter()          # mais preciso que time.time()
resultado = busca_linear(lista, alvo)
fim = time.perf_counter()
print(f"{{(fim - inicio) * 1000:.4f}} ms")

# Melhor: repetir e usar o mínimo ou a mediana (ignora picos do sistema operacional)
tempos = timeit.repeat(lambda: fib_memo(30), repeat=7, number=100)
print(min(tempos) / 100, statistics.median(tempos) / 100)

random.seed(42)                       # reprodutibilidade: mesmos sorteios em toda execução
```

PARTE 3 — Exercícios com respostas

Tema 1 — Notação Big O e crescimento

1.1 Explique, em suas palavras, o que a notação Big O mede e o que ela ignora.

Resposta. Mede a **taxa de crescimento** do número de operações em função do tamanho da entrada n , normalmente no **pior caso**. Ignora constantes multiplicativas e termos de ordem menor ($3n^2 + 5n$ é $O(n^2)$), e não diz o tempo em segundos — dois algoritmos $O(n)$ podem ter velocidades bem diferentes. Serve para comparar como os algoritmos **escalam** quando n cresce.

1.2 Ordene do mais rápido ao mais lento: $O(n!)$, $O(1)$, $O(n^2)$, $O(\log n)$, $O(2^n)$, $O(n \log n)$, $O(n)$.

Resposta. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$.

1.3 Simplifique para a notação Big O: (a) $3n^2 + 5n + 10$ · (b) $20 \cdot \log n + 4$ · (c) $n/26$ · (d) $2n + n \cdot \log n$ · (e) 500 · (f) $n^2 + 2^n$ · (g) $n(n-1)/2$.

Resposta. (a) $O(n^2)$ · (b) $O(\log n)$ · (c) $O(n)$ — dividir por 26 é uma constante · (d) $O(n \log n)$ — fica o termo dominante · (e) $O(1)$ · (f) $O(2^n)$ · (g) $O(n^2)$ — $n(n-1)/2 = (n^2 - n)/2$.

1.4 Se o tamanho da entrada **dobra**, quantas vezes o trabalho cresce em um algoritmo $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$ e $O(2^n)$?

Resposta. $O(1)$: não muda. $O(\log n)$: apenas **+1 passo** ($\log_2 2n = \log_2 n + 1$). $O(n)$: $\times 2$. $O(n \log n)$: $2n \cdot \log(2n) = 2n \cdot \log n + 2n \rightarrow$ um pouco mais que $\times 2$. $O(n^2)$: $(2n)^2 = 4n^2 \rightarrow \times 4$. $O(2^n)$: $2^{(2n)} = (2^n)^2 \rightarrow$ o trabalho é **elevado ao quadrado** (e cada +1 em n já dobra).

Isso é exatamente o que a tabela da atividade 02 mostrou: de $n = 50$ para $n = 100$, as comparações do bubble sort no pior caso foram de 1.225 para 4.950 ($\approx 4\times$).

1.5 Um algoritmo $O(n^2)$ leva 2 s para $n = 1.000$. Estime o tempo para $n = 10.000$. E se ele fosse $O(n \log n)$?

Resposta. n cresceu $10\times$. Em $O(n^2)$ o trabalho cresce $10^2 = 100\times \rightarrow \approx 200$ s. Em $O(n \log n)$: $(10.000 \cdot \log_2 10.000) / (1.000 \cdot \log_2 1.000) = (10.000 \cdot 13,3) / (1.000 \cdot 9,97) \approx 13,3\times \rightarrow \approx 27$ s. É uma estimativa: Big O ignora constantes, então o valor real pode variar, mas a ordem de grandeza é essa.

1.6 Dê o Big O de cada trecho:

```
# (a)
for i in range(n):
    print(lista[i])

# (b)
for i in range(n):
    for j in range(n):
        print(i * j)

# (c)
i = n
while i > 1:
    i = i // 2
```

```
# (d)
print(lista[0])
print(lista[-1])

# (e)
for i in range(n):
    for j in range(i + 1, n):
        comparar(lista[i], lista[j])

# (f)
for i in range(n):
    j = 1
    while j < n:
        j = j * 2
```

Resposta. (a) $O(n)$ · (b) $O(n^2)$ · (c) $O(\log n)$ — divide n pela metade a cada volta, como a busca binária · (d) $O(1)$ — duas operações fixas, independentes de n · (e) $O(n^2)$ — são $n(n-1)/2$ pares, o padrão do selection sort e do bubble sort · (f) $O(n \log n)$ — laço externo n vezes, interno $\log n$ vezes.

1.7 Verdadeiro ou falso, justificando: “Um algoritmo $O(\log n)$ é sempre mais rápido que um $O(n)$, para qualquer entrada.”

Resposta. Falso. Big O descreve o crescimento no pior caso, não cada instância. No **melhor caso** da busca (alvo na 1ª posição), a busca linear $O(n)$ acha em 1 passo, e a binária $O(\log n)$ ainda precisa afunilar até o índice 0 (vários passos). Além disso, para n pequeno as constantes escondidas podem inverter o resultado. Isso **não invalida** o $O(\log n)$: quando n cresce, a binária domina — no experimento da atividade 01, com 1 milhão de itens, foram 20 passos contra 1.000.000.

1.8 (Livro, cap. 1) O problema do caixeiro-viajante é $O(n!)$. Quantas rotas há para 5, 6, 7 e 10 cidades? Por que esse crescimento é considerado inviável?

Resposta. $5! = 120$ · $6! = 720$ · $7! = 5.040$ · $10! = 3.628.800$. Cada cidade a mais **multiplica** o trabalho por n : com 20 cidades já são $\approx 2,4 \times 10^{18}$ rotas. Nenhum computador resolve por força bruta para n grande; é o exemplo do livro de algoritmo “terrivelmente lento” para o qual não se conhece solução rápida.

1.9 (Livro, cap. 1) Complete a tabela para um computador que faz **10 operações por segundo** e uma lista de **$n = 16$** itens.

Resposta. $O(\log n) = 4$ operações $\rightarrow 0,4$ s · $O(n) = 16 \rightarrow 1,6$ s · $O(n \log n) = 64 \rightarrow 6,4$ s · $O(n^2) = 256 \rightarrow 25,6$ s · $O(n!) = 16! \approx 2,09 \times 10^{13} \rightarrow \approx 66$ mil anos.

1.10 Por que a análise Big O costuma falar do **pior caso**? O que são o melhor caso e o caso médio?

Resposta. O pior caso dá uma **garantia**: “nunca será mais lento que isso”, o que importa para dimensionar sistemas. O **melhor caso** é a entrada mais favorável (lista já ordenada para o bubble sort; alvo na 1ª posição para a busca linear). O **caso médio** é o comportamento esperado em entradas típicas/aleatórias (quicksort: $O(n \log n)$ no médio, $O(n^2)$ no pior). Para alguns algoritmos os três coincidem (selection sort: sempre $O(n^2)$); para outros diferem muito (bubble sort: $O(n)$ a $O(n^2)$; quicksort: $O(n \log n)$ a $O(n^2)$).

1.11 Complexidade de **tempo** e de **espaço** são a mesma coisa? Dê um exemplo em que elas divergem.

Resposta. Não. Tempo conta operações; espaço conta memória extra usada (incluindo a **pilha de chamadas**). O Fibonacci recursivo puro gasta tempo **exponencial** $O(2^n)$, mas espaço apenas **$O(n)$** : a árvore é percorrida em profundidade, então existem no máximo n chamadas empilhadas ao mesmo tempo — as chamadas são muitas, mas não simultâneas. Já o merge sort é $O(n \log n)$ no tempo e $O(n)$ no espaço, enquanto o bubble sort é $O(n^2)$ no tempo e $O(1)$ no espaço.

Tema 2 — Busca linear × busca binária

2.1 (Livro, ex. 1.1) Você tem uma lista **ordenada** de 128 nomes e procura um deles com busca binária. Qual o número máximo de passos?

Resposta. 7, pois $\log_2 128 = 7$ ($2^7 = 128$). A cada passo a lista cai pela metade: $128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$. *Observação:* uma implementação que conta a comparação final também dá $\lfloor \log_2 n \rfloor + 1 = 8$ no pior caso. Na prova, use a conta do livro: $\lceil \log_2 n \rceil$.

2.2 (Livro, ex. 1.2) Suponha que a lista dobre de tamanho (256 nomes). Qual o número máximo de passos agora?

Resposta. 8. Dobrar a lista acrescenta apenas **um** passo: essa é a marca de $O(\log n)$.

2.3 Uma lista tem 1.000.000 de números ordenados e o alvo é o último. Quantos passos faz a busca linear? E a binária? Quantas vezes a binária faz menos passos?

Resposta. Linear: **1.000.000** passos (olha a lista toda). Binária: $\log_2 1.000.000 \approx 19,93 \rightarrow$ **20 passos** ($2^{20} = 1.048.576 \geq 1.000.000$). A binária faz **50.000 vezes menos passos**. Foi exatamente o resultado da atividade 01 ($1.000.000 \times 20$ passos; em tempo, $\approx 33 \text{ ms} \times 0,014 \text{ ms}$).

2.4 (Livro, cap. 1) Um dicionário tem 240.000 palavras. Quantos passos, no pior caso, a busca binária leva? E em uma lista com 4 bilhões de itens?

Resposta. $2^{17} = 131.072 < 240.000 \leq 262.144 = 2^{18} \rightarrow$ **18 passos**. Para 4 bilhões: $2^{32} \approx 4,29 \text{ bilhões} \rightarrow$ **32 passos**. A busca linear precisaria de até 4 bilhões.

2.5 Por que a busca binária **exige** que a lista esteja ordenada?

Resposta. Porque a cada passo ela compara o alvo com o elemento do meio e **descarta uma metade inteira** com base nessa comparação (“é maior, então só pode estar à direita”). Isso só é válido se tudo à esquerda for menor e tudo à direita for maior — ou seja, se a lista estiver ordenada. Em lista desordenada, a comparação com o meio não diz nada sobre onde o alvo está, e o algoritmo pode descartar a metade que contém a resposta.

2.6 Simule a busca binária de **37** em $[3, 8, 12, 19, 25, 31, 37, 42, 49, 56]$ (índices 0 a 9), mostrando baixo, alto, meio e o chute a cada passo. Quantos passos? Quantos faria a busca linear?

Resposta.

Passo	baixo	alto	meio	lista[meio]	Ação
1	0	9	4	25	$25 < 37 \rightarrow \text{baixo} = 5$
2	5	9	7	42	$42 > 37 \rightarrow \text{alto} = 6$
3	5	6	5	31	$31 < 37 \rightarrow \text{baixo} = 6$
4	6	6	6	37	achou no índice 6

Binária: **4 passos**. Linear: **7 passos** (o 37 é o 7º elemento). Com $n = 10$, $\lfloor \log_2 10 \rfloor + 1 = 4$ é o máximo possível para a binária.

2.7 Na mesma lista, simule a busca binária de **4** (um valor que **não** está na lista). Como o algoritmo descobre que o item não existe?

Resposta. Passo 1: meio = 4 (25) $\rightarrow 25 > 4 \rightarrow \text{alto} = 3$. Passo 2: meio = 1 (8) $\rightarrow 8 > 4 \rightarrow \text{alto} = 0$. Passo 3: meio = 0 (3) $\rightarrow 3 < 4 \rightarrow \text{baixo} = 1$. Agora baixo (1) $>$ alto (0): o intervalo ficou vazio, o laço termina e a função devolve None após **3 passos**. A busca linear precisaria de todos os 10 passos para concluir que o 4 não está lá.

2.8 (Livro, ex. 1.3 a 1.6) Dê o Big O de cada operação em uma **agenda telefônica ordenada por nome**: (a) você tem o nome e quer o telefone; (b) você tem o telefone e quer o nome; (c) você quer ler o telefone de todas as pessoas; (d) você quer ler o telefone só das pessoas cujo nome começa com A.

Resposta. (a) **$O(\log n)$** — busca binária pelo nome, a agenda está ordenada por nome. (b) **$O(n)$** — a agenda não está ordenada por telefone, então é preciso varrer tudo (busca linear). (c) **$O(n)$** — lê cada entrada uma vez. (d) **$O(n)$** — parece “só 1/26 da lista”, mas Big O ignora constantes: $O(n/26) = O(n)$. (O próprio livro avisa que essa resposta surpreende.)

2.9 Você tem uma lista **desordenada** e vai fazer **uma única** busca. Vale ordenar a lista e usar busca binária? E se forem milhares de buscas?

Resposta. Para uma busca, **não**: ordenar custa $O(n \log n)$, mais caro que uma busca linear $O(n)$ direta. Para muitas buscas, **sim**: paga-se $O(n \log n)$ uma vez e depois cada busca custa $O(\log n)$ em vez de $O(n)$. Existe um **ponto de equilíbrio** (no experimento avançado da atividade 01, cerca de 12 buscas): abaixo dele a linear vence; acima, ordenar uma vez compensa. A resposta para “qual é melhor?” é “**depende de quantas vezes você vai buscar**”.

2.10 Escreva, com suas palavras, a diferença entre $O(n)$ e $O(\log n)$. (Pergunta da atividade 01.)

Resposta modelo. Em $O(n)$ o trabalho cresce **na mesma proporção** que a entrada: 10 vezes mais itens, 10 vezes mais passos — é o caso da busca linear, que olha um item por vez. Em $O(\log n)$ o trabalho cresce **muito mais devagar**: cada vez que a entrada **dobra**, aumenta só **um passo** — é o caso da busca binária, que corta a lista pela metade a cada comparação. Para 1 milhão de itens: 1.000.000 passos contra 20. A diferença aparece no pior caso e fica cada vez maior conforme n cresce.

2.11 No experimento da atividade 01, a binária fez 50.000× menos passos (20 contra 1.000.000), mas foi “só” $\approx 2.300\times$ mais rápida em tempo. Por que a razão de tempo é menor que a razão de passos?

Resposta. Porque **passos não são segundos**. Cada passo tem um custo diferente: o passo da binária faz uma divisão inteira, uma comparação e atualiza dois índices, enquanto o passo da linear é uma comparação simples dentro de um for muito otimizado. Além disso há custo fixo de chamar a função e a resolução do relógio pesa em medições de 0,01 ms. Isso mostra por que se mede **passos e tempo**: a contagem prova a classe de complexidade; o tempo mostra o efeito real, com as constantes que Big O ignora.

2.12 Explique o melhor caso das duas buscas. Qual delas vence quando o alvo está na posição 0?

Resposta. Busca linear: melhor caso é o alvo na **1ª posição**: 1 passo, $O(1)$. Busca binária: melhor caso é o alvo exatamente **no meio**: 1 passo. Se o alvo está na posição 0, a linear acha em 1 passo, mas a binária precisa de $\approx \log_2 n$ passos (vai afunilando até o índice 0). Ou seja, **a linear vence nesse caso particular**, o que não contradiz o Big O — ele descreve crescimento e pior caso.

Tema 3 — Ordenação: selection sort e bubble sort

3.1 (Livro, cap. 2) Descreva o selection sort. Por que ele é $O(n^2)$ e não $O(n^2/2)$?

Resposta. A cada passagem, ele percorre o trecho ainda não ordenado, **encontra o menor** e o coloca na primeira posição livre. Para achar o menor de n itens faz $n-1$ comparações, depois $n-2$, depois $n-3$... total $n(n-1)/2 \approx n^2/2$ comparações. Big O **ignora constantes** como o $1/2$, então fica **$O(n^2)$** .

3.2 Simule o **selection sort** em [5, 2, 9, 1, 7], mostrando a lista após cada passagem. Conte comparações e trocas.

Resposta.

Passagem	Menor encontrado	Lista após a passagem	Comparações
1	1 (índice 3) $\leftrightarrow$ posição 0	[1, 2, 9, 5, 7]	4
2	2 (já está na posição 1)	[1, 2, 9, 5, 7]	3
3	5 (índice 3) $\leftrightarrow$ posição 2	[1, 2, 5, 9, 7]	2
4	7 (índice 4) $\leftrightarrow$ posição 3	[1, 2, 5, 7, 9]	1

Total: **10 comparações** ($= 5 \cdot 4/2$) e **3 trocas** efetivas (na passagem 2 o menor já estava no lugar).

3.3 Simule o **bubble sort** (com parada quando não há troca) em [5, 2, 9, 1, 7], mostrando cada passagem. Conte comparações, trocas e passagens.

Resposta.

Passagem	Comparações (pares)	Trocas	Lista ao final
1	(5,2) troca · (5,9) · (9,1) troca · (9,7) troca	3	[2, 5, 1, 7, 9]
2	(2,5) · (5,1) troca · (5,7)	1	[2, 1, 5, 7, 9]
3	(2,1) troca · (2,5)	1	[1, 2, 5, 7, 9]
4	(1,2)	0	[1, 2, 5, 7, 9] → sem troca: para

Total: **10 comparações** ($4+3+2+1$), **5 trocas**, **4 passagens**. Note que o maior elemento “sobe” para o fim a cada passagem (9, depois 7...), por isso a passagem seguinte compara um par a menos.

3.4 Para $n = 5, 10, 100$ e 500 , quantas comparações fazem o selection sort (sempre) e o bubble sort (pior caso)? E quantas trocas o bubble sort faz no pior caso?

Resposta. $n(n-1)/2$: $n = 5 \rightarrow 10$; $n = 10 \rightarrow 45$; $n = 100 \rightarrow 4.950$; $n = 500 \rightarrow 124.750$. No pior caso (lista invertida) o bubble sort troca em **toda** comparação, então as trocas também são $n(n-1)/2$ (10, 45, 4.950, 124.750) — foi exatamente a tabela da atividade 02.

3.5 Com a lista $[1, 2, 3, 4, 5]$ (já ordenada), quantas comparações e trocas fazem o bubble sort com parada e o selection sort? O que isso diz sobre o melhor caso de cada um?

Resposta. Bubble sort: **4 comparações, 0 trocas, 1 passagem** — percebe que não houve troca e para $\rightarrow$ melhor caso $O(n)$. Selection sort: **10 comparações, 0 trocas** — ele não aproveita a ordem existente, sempre varre o trecho restante para achar o menor $\rightarrow O(n^2)$ mesmo no melhor caso.

3.6 Com a lista invertida $[5, 4, 3, 2, 1]$, quantas comparações e trocas faz cada algoritmo?

Resposta. Bubble sort: **10 comparações e 10 trocas** (pior caso: todo par comparado está fora de ordem). Passagens: $[4,3,2,1,5] \rightarrow [3,2,1,4,5] \rightarrow [2,1,3,4,5] \rightarrow [1,2,3,4,5]$. Selection sort: **10 comparações e apenas 2 trocas** ($5 \leftrightarrow 1 \rightarrow [1,4,3,2,5]$; $4 \leftrightarrow 2 \rightarrow [1,2,3,4,5]$; as demais passagens não trocam).

3.7 Na atividade 02, quando n foi de 50 para 100, as comparações do pior caso foram de 1.225 para 4.950. Que fator é esse e o que ele confirma?

Resposta. $4.950 / 1.225 \approx 4$. Dobrar n quadruplicou o trabalho: $2^2 = 4$, comportamento **quadrático**, confirmando $O(n^2)$ experimentalmente.

3.8 Em que situação o bubble sort é a melhor escolha? E o selection sort? E quando nenhum dos dois?

Resposta. **Bubble sort:** lista **quase ordenada** — ele detecta a ordem e para cedo (tende a $O(n)$). **Selection sort:** quando **a troca é cara** (registros grandes, escrita em disco), pois faz no máximo $n-1$ trocas, contra até $n(n-1)/2$ do bubble sort. **Nenhum dos dois:** listas grandes — ambos são $O(n^2)$; use um $O(n \log n)$ como quicksort ou merge sort. Para $n = 1.000.000$, $n^2/2 \approx 5 \times 10^{11}$ comparações contra $n \cdot \log_2 n \approx 2 \times 10^7$ (≈ 25.000 vezes menos).

3.9 Por que o número de operações do bubble sort **varia** com a entrada e o do selection sort **não**?

Resposta. O bubble sort só troca quando encontra um par vizinho fora de ordem e **para quando uma passagem inteira não troca nada**; seu trabalho depende de **quanto** a lista já está ordenada. O selection sort sempre percorre **todo** o trecho restante para achar o menor, sem olhar se a lista já está em ordem, por isso faz sempre $n(n-1)/2$ comparações. As trocas do selection sort variam um pouco (0 a $n-1$), mas as comparações não.

3.10 (Livro, cap. 2 — extra) Complete a tabela de arrays $\times$ listas encadeadas e responda os exercícios 2.1 a 2.5 do livro.

Resposta.

Operação	Array	Lista encadeada
Leitura (acesso pelo índice)	$O(1)$	$O(n)$
Inserção	$O(n)$	$O(1)$
Remoção	$O(n)$	$O(1)$

(Inserção/remoção na lista encadeada é $O(1)$ supondo que você já está no elemento; array permite acesso aleatório, a lista só sequencial.)

2.1 App de finanças: muitas **inserções**, poucas leituras (e sempre lê tudo em sequência) → **lista encadeada**. **2.2** Fila de pedidos de restaurante: garçons inserem no fim, cozinheiros retiram do início; não precisa de acesso aleatório → **lista encadeada**. **2.3** Login no Facebook: busca frequente pelo nome em uma lista **ordenada** → **array**, porque a busca binária exige acesso aleatório ($O(1)$ ao meio). **2.4** Desvantagens do array para inserção: manter a ordem exige **deslocar** elementos ($O(n)$) e, se o array encher, é preciso **realocar** tudo em um espaço maior. **2.5** Híbrido: array de 26 posições (A–Z), cada uma apontando para uma lista encadeada. **Busca:** mais lenta que array (não dá para fazer busca binária completa), mais rápida que lista pura (só varre a letra certa). **Inserção:** mais rápida que array, igual à lista encadeada. É a semente da tabela hash.

Tema 4 — Recursividade

4.1 O que são **caso base** e **caso recursivo**? (Livro, ex. 3.2) O que acontece se você esquecer o caso base?

Resposta. Caso base: a condição em que a função **não chama a si mesma** e devolve uma resposta direta (lista vazia → 0; $n < 2 \rightarrow n$). **Caso recursivo:** a função chama a si mesma com um problema **menor**, aproximando-se do caso base. Sem caso base a função nunca para: cada chamada empilha um novo quadro, a pilha cresce até acabar a memória reservada — **stack overflow**; em Python, `RecursionError: maximum recursion depth exceeded` (limite padrão ≈ 1.000 chamadas; a aula 06 usou `sys.setrecursionlimit(2000)`).

4.2 (Livro, ex. 3.1) Suponha que a pilha de chamadas esteja assim (topo em cima):

```
[ greet2 | name: maggie ]
[ greet  | name: maggie ]
```

Que informações você consegue extrair só olhando a pilha?

Resposta. (1) greet foi chamada primeiro, com `name = "maggie"`; (2) greet chamou greet2, também com `name = "maggie"`; (3) greet está **incompleta/suspensa**, esperando greet2 retornar; (4) quando greet2 terminar, seu quadro sai da pilha e greet continua de onde parou. A pilha guarda as variáveis de cada chamada e a ordem de retorno (a última que entrou é a primeira que sai).

4.3 Trace a execução de `soma([2, 4, 6])` (função da aula 06). Qual é a profundidade máxima da pilha?

Resposta.

```
soma([2, 4, 6])  
= 2 + soma([4, 6])  
= 2 + (4 + soma([6]))  
= 2 + (4 + (6 + soma([]))) ← caso base: soma([]) = 0  
= 2 + (4 + (6 + 0)) = 12
```

Profundidade máxima: **4 quadros** empilhados ao mesmo tempo (as três chamadas com lista não vazia mais a do caso base). Em geral, para uma lista de n elementos são $n+1$ chamadas $\rightarrow O(n)$ tempo e $O(n)$ de pilha.

4.4 Escreva funções **recursivas** para: (a) contar os itens de uma lista; (b) achar o maior valor de uma lista; (c) fatorial de n . Indique o caso base de cada uma.

Resposta. Código na Parte 2, seção 2.4. Casos base: (a) lista vazia $\rightarrow 0$; (b) lista com 1 elemento $\rightarrow$ esse elemento; (c) $n = 1 \rightarrow 1$ (ou $n = 0 \rightarrow 1$). Caso recursivo: (a) $1 + \text{contar}(\text{resto})$; (b) o maior entre o primeiro e o máximo do resto; (c) $n \times \text{fatorial}(n-1)$.

4.5 Quantas chamadas e qual a profundidade da pilha em `fatorial(5)`? Mostre a ordem em que os resultados são calculados.

Resposta. 5 chamadas (`fatorial(5)  $\rightarrow$  (4)  $\rightarrow$  (3)  $\rightarrow$  (2)  $\rightarrow$  (1)`), profundidade **5**. Os retornos acontecem **de baixo para cima**: `fatorial(1) = 1  $\rightarrow$  fatorial(2) = 2·1 = 2  $\rightarrow$  fatorial(3) = 3·2 = 6  $\rightarrow$  fatorial(4) = 4·6 = 24  $\rightarrow$  fatorial(5) = 5·24 = 120`. Nada é multiplicado antes de chegar ao caso base.

4.6 O que esta função imprime para `regressiva(3)`? Qual é o caso base?

```
def regressiva(i):  
    print(i)  
    if i <= 1:  
        return  
    regressiva(i - 1)
```

Resposta. Imprime 3, 2, 1 (um por linha). Caso base: $i \leq 1$ (imprime e retorna sem chamar de novo). Se o `if` fosse removido, imprimiria 3, 2, 1, 0, -1, ... até estourar a pilha.

4.7 “Laços podem melhorar o desempenho do programa; recursão pode melhorar o desempenho do programador.” Explique a frase (citada no livro) com um exemplo do curso.

Resposta. Recursão não é mais rápida: cada chamada custa tempo e memória de pilha (e pode estourar). Ela é usada porque deixa a solução **mais clara**, especialmente em problemas naturalmente recursivos (dividir para conquistar). Exemplo do curso: Fibonacci recursivo puro é elegante mas $O(2^n)$; o iterativo é $O(n)$ e $O(1)$ de memória. Já o quicksort é muito mais simples de escrever recursivamente.

4.8 O Fibonacci recursivo puro faz $\approx 2,7$ milhões de chamadas para $n = 30$. Isso significa que ele usa 2,7 milhões de posições na pilha? Explique.

Resposta. Não. A árvore de chamadas é percorrida **em profundidade**: quando `fib(29)` termina, seu quadro sai antes de `fib(28)` começar. Em qualquer instante há no máximo **n** quadros empilhados (o caminho da raiz até um caso base). Tempo $O(2^n)$, pilha $O(n)$. As chamadas são muitas, mas **não simultâneas**.

Tema 5 — Fibonacci: recursivo puro × memorização (cache)

5.1 Defina a sequência de Fibonacci e liste $F(0)$ a $F(15)$.

Resposta. $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$. 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610.

5.2 Desenhe a árvore de chamadas de `fib_recursivo(5)`. Quantas chamadas ocorrem? Quantas vezes `fib(2)` e `fib(1)` são recalculadas?

Resposta.

```
fib(5)
├─ fib(4)
│   ├── fib(3)
│   │   ├── fib(2)
│   │   │   ├── fib(1)
│   │   │   └─ fib(0)
│   │   └─ fib(1)
│   └─ fib(2)
│       ├── fib(1)
│       └─ fib(0)
└─ fib(3)
    ├── fib(2)
    │   ├── fib(1)
    │   └─ fib(0)
    └─ fib(1)
```

15 chamadas no total. `fib(3)` é calculado 2×, `fib(2)` 3×, `fib(1)` 5×, `fib(0)` 3×. Esse recálculo de **subproblemas sobrepostos** é o desperdício que a memorização elimina.

5.3 Por que a abordagem recursiva “pura” não é ideal? Em que a memorização ajuda? (Pergunta 1 da atividade 03.)

Resposta. Porque ela **recalcula os mesmos subproblemas** muitas vezes: `fib(n)` chama `fib(n-1)` e `fib(n-2)`, e `fib(n-1)` chama `fib(n-2)` de novo, e assim por diante — a árvore de chamadas cresce **exponencialmente** ($\approx 1,618^n$). A memorização (cache) guarda o resultado de cada `fib(k)` na primeira vez que ele é calculado (em um dict ou com `@lru_cache`); nas próximas vezes a função só **consulta** o cache em $O(1)$. Cada subproblema é resolvido **uma única vez** → o total cai para linear.

5.4 Qual é a complexidade de cada abordagem e como se chega a essa conclusão? (Pergunta 2 da atividade 03.)

Resposta. Recursiva pura: seja $T(n)$ o número de chamadas; lendo o código, $T(n) = T(n-1) + T(n-2) + 1$, com $T(0) = T(1) = 1$. A solução é $T(n) = 2 \cdot F(n+1) - 1$, e como $F(n)$ cresce como $\phi^n / \sqrt{5}$ ($\phi \approx 1,618$), $T(n) = \Theta(\phi^n) \subset O(2^n)$ — exponencial. Confirmação experimental: 15, 177, 21.891, 2.692.537 chamadas para $n = 5, 10, 20, 30$; a razão $T(n)/T(n-1)$ converge para 1,618. **Memoizada:** há no máximo $n+1$ chaves no cache, cada uma calculada uma vez (2 chamadas cada) → exatamente $2n - 1$ chamadas → $O(n)$; 9, 19, 39, 59 chamadas para $n = 5, 10, 20, 30$. **Iterativa:** um laço de n voltas → $O(n)$, com memória $O(1)$.

5.5 É possível decidir qual é mais eficiente usando outros parâmetros, como tempo e memória? (Pergunta 3 da atividade 03.)

Resposta. Sim, e é importante medir os dois. **Tempo:** na atividade, para $n = 32$ a recursiva pura levou $\approx 0,54$ s e a memoizada $\approx 0,000006$ s ($\approx 88.000\times$ mais rápida); a curva da recursiva sobe em linha reta em escala logarítmica (exponencial), as outras ficam quase planas. **Memória:** a recursiva pura usa pilha $O(n)$ e nada mais; a memoizada usa **cache $O(n)$** mais pilha $O(n)$ (pode dar RecursionError para n na casa dos milhares); a iterativa usa **$O(1)$** . Conclusão: memoizada troca memória por tempo; iterativa é a melhor nos dois critérios; recursiva pura só é aceitável para n pequeno (≤ 30).

5.6 Se `fib_recursivo(30)` leva $\approx 0,13$ s, estime `fib_recursivo(40)`. E `fib_recursivo(31)`?

Resposta. A cada $+1$ em n o trabalho multiplica por $\phi \approx 1,618$. Então `fib(31)` $\approx 0,13 \times 1,618 \approx$ **$0,21$ s**, e `fib(40)` $\approx 0,13 \times 1,618^{10} \approx 0,13 \times 123 \approx$ **16 s**. (Para $n = 50$ seriam horas.)

5.7 Por que a versão memoizada **top-down** ainda pode falhar com RecursionError para $n = 5.000$, e a iterativa não?

Resposta. A memoizada continua sendo **recursiva**: a primeira chamada `fib_memo(5000)` desce até `fib_memo(1)` antes de qualquer valor entrar no cache, empilhando ≈ 5.000 quadros — acima do limite padrão do Python (≈ 1.000). A iterativa (bottom-up) usa apenas duas variáveis e um laço: pilha constante.

5.8 O que é um “cache quente” e por que a segunda chamada de `fib_lru(300)` custa nanossegundos?

Resposta. Cache quente é o cache **já preenchido** por uma chamada anterior. Na segunda chamada, `fib_lru(300)` encontra o valor pronto e retorna em **$O(1)$** (≈ 56 ns na atividade). É por isso que, em benchmarks honestos, o cache precisa ser **zerado** entre medições — senão você mede uma consulta, não o algoritmo.

5.9 Complete a tabela:

n	Chamadas recursivo puro	Chamadas memoizado	Quantas vezes menos
5			
10			
20			
30			

Resposta. $n = 5$: $15 \cdot 9 \cdot \approx 2\times$ — $n = 10$: $177 \cdot 19 \cdot \approx 9\times$ — $n = 20$: $21.891 \cdot 39 \cdot \approx 561\times$ — $n = 30$: $2.692.537 \cdot 59 \cdot \approx 45.636\times$. (Fórmulas: $2 \cdot F(n+1) - 1$ e $2n - 1$.)

Tema 6 — Dividir para conquistar e Quicksort

6.1 Defina a estratégia “dividir para conquistar” e suas três etapas. Cite três algoritmos do curso que a usam.

Resposta. Resolver um problema quebrando-o em **subproblemas menores do mesmo tipo**, resolvendo-os recursivamente e juntando os resultados. Etapas: **Dividir** → **Conquistar** (resolver recursivamente, até o caso base) → **Combinar**. Exemplos: **quicksort** (particiona em torno do pivô e concatena), **merge sort** (divide ao meio e intercala), **busca binária** (descarta uma metade a cada passo) e o **maior quadrado da fazenda** (algoritmo de Euclides).

6.2 (Livro, cap. 4 — exemplo da aula) Um terreno mede 1680×640 m e deve ser dividido em lotes **quadrados iguais** do **maior** tamanho possível. Mostre a sequência de reduções até o caso base e o resultado.

Resposta. Caso base: um lado é múltiplo do outro. Caso recursivo: preencha o retângulo com o maior quadrado possível e aplique o mesmo algoritmo à **sobra**. $(1680, 640) \rightarrow$ sobra $1680 \bmod 640 = 400 \rightarrow (640, 400) \rightarrow$ sobra $240 \rightarrow (400, 240) \rightarrow$ sobra $160 \rightarrow (240, 160) \rightarrow$ sobra $80 \rightarrow (160, 80)$: 160 é múltiplo de 80 $\rightarrow$ **caso base**. Resposta: lotes de **80×80 m**. O maior quadrado que cabe na sobra é o maior que serve para a fazenda inteira. São 5 chamadas de maior_quadrado. É o algoritmo de Euclides para o $\text{MDC}(1680, 640) = 80$.

6.3 Aplique maior_quadrado(1200, 450) e maior_quadrado(1071, 462), mostrando as chamadas.

Resposta. $(1200, 450) \rightarrow 1200 \bmod 450 = 300 \rightarrow (450, 300) \rightarrow 450 \bmod 300 = 150 \rightarrow (300, 150)$: $300 \bmod 150 = 0 \rightarrow$ **150 m** (3 chamadas). $(1071, 462) \rightarrow 1071 \bmod 462 = 147 \rightarrow (462, 147) \rightarrow 462 \bmod 147 = 21 \rightarrow (147, 21)$: $147 \bmod 21 = 0 \rightarrow$ **21** (3 chamadas).

6.4 (Livro, ex. 4.1 a 4.3) Escreva de forma recursiva: soma de uma lista; contagem de itens; maior valor. Identifique caso base e caso recursivo.

Resposta. Código na Parte 2, seção 2.4. Em todos, o caso recursivo trabalha com `lista[1:]` (a lista menos o primeiro elemento), que é o problema **reduzido**; o caso base é a lista vazia (soma = 0, contagem = 0) ou com um único elemento (máximo = ele mesmo).

6.5 (Livro, ex. 4.4) A busca binária também é dividir para conquistar. Descreva seu caso base e seu caso recursivo.

Resposta. Caso base: o intervalo tem 1 elemento (ou ficou vazio): se for o alvo, achou; senão, o item não está na lista. **Caso recursivo:** compare o alvo com o elemento do meio, **descarte a metade** que não pode conter o alvo e chame a busca binária na metade restante. Código em 2.1 (`busca_binaria_rec`).

6.6 (Livro, ex. 4.5 a 4.8) Dê o Big O de: (a) imprimir cada elemento de um array; (b) dobrar o valor de cada elemento; (c) dobrar o valor **apenas do primeiro** elemento; (d) criar a tabela de multiplicação de todos os elementos (para $[2, 3, 7, 8, 10]$, multiplicar cada um por 2, depois por 3, por 7...).

Resposta. (a) $O(n)$ · (b) $O(n)$ · (c) $O(1)$ · (d) $O(n^2)$ — n linhas $\times$ n colunas.

6.7 Simule o quicksort da aula (pivô = **primeiro** elemento) em $[33, 15, 10, 42, 7]$. Mostre pivô, subarray dos menores e dos maiores em cada chamada, e a combinação final. Quantas chamadas de quicksort ocorrem e qual a altura da pilha?

Resposta.

```
quicksort([33, 15, 10, 42, 7])  pivô 33 → menores [15, 10, 7] · maiores [42]
├─ quicksort([15, 10, 7])      pivô 15 → menores [10, 7]  · maiores []
│   ├─ quicksort([10, 7])     pivô 10 → menores [7]      · maiores []
│   │   └─ quicksort([7])     → [7] (caso base)
│   │   └─ quicksort([])      → [] (caso base)
│   │   └─ combina: [7] + [10] + [] = [7, 10]
│   └─ quicksort([])          → [] (caso base)
├─ combina: [7, 10] + [15] + [] = [7, 10, 15]
└─ quicksort([42])            → [42] (caso base)
combina: [7, 10, 15] + [33] + [42] = [7, 10, 15, 33, 42]
```

7 chamadas de quicksort, altura da pilha **4** níveis (raiz → [15,10,7] → [10,7] → [7]).

6.8 Simule o quicksort com pivô no primeiro elemento em [8, 3, 5, 1, 9, 2].

Resposta. pivô 8 → [3, 5, 1, 2] · 8 · [9]. Em [3, 5, 1, 2]: pivô 3 → [1, 2] · 3 · [5]. Em [1, 2]: pivô 1 → [] · 1 · [2].
Combinando: [1, 2] → [1, 2, 3, 5] → **[1, 2, 3, 5, 8, 9]**. 7 chamadas, altura 4.

6.9 Qual é o **pior caso** do quicksort? Mostre com [1, 2, 3, 4, 5] e pivô = primeiro elemento. Por que o custo total vira $O(n^2)$?

Resposta. Pior caso: lista **já ordenada** (ou invertida) com pivô fixo no primeiro elemento. pivô 1 → [] · 1 · [2,3,4,5]; pivô 2 → [] · 2 · [3,4,5]; pivô 3 → [] · 3 · [4,5]; pivô 4 → [] · 4 · [5]. Um dos subarrays é **sempre vazio**: a lista não é dividida ao meio, só diminui de 1. A pilha fica com **altura n** (5 níveis) e cada nível ainda toca todos os elementos restantes ($4 + 3 + 2 + 1 = 10$ comparações = $n(n-1)/2$). Custo: $O(n)$ por nível × n níveis = **$O(n^2)$** — tão lento quanto o selection sort.

6.10 Qual é o **melhor caso**? Mostre com [1, 2, 3, 4, 5, 6, 7] e pivô no **meio**. Por que o custo total é $O(n \log n)$?

Resposta. pivô 4 → [1, 2, 3] · 4 · [5, 6, 7]; em cada metade, pivô 2 → [1] · 2 · [3] e pivô 6 → [5] · 6 · [7]. A lista é dividida **ao meio** a cada nível, então a altura da pilha é $\approx \log_2 n$ (3 níveis para $n = 7$). Cada nível toca $O(n)$ elementos → **$O(n) \times O(\log n) = O(n \log n)$** .

6.11 Por que o custo de **cada nível** da recursão do quicksort é $O(n)$, independentemente do pivô escolhido?

Resposta. Porque em cada nível o particionamento percorre **todos os elementos** que chegaram àquele nível para separá-los em menores/maiores que o pivô. Somando os subarrays de um mesmo nível, sempre há $\approx n$ elementos. O pivô **não muda o custo do nível**; ele muda a **quantidade de níveis** (altura da pilha): $\log n$ no melhor caso, n no pior.

6.12 Como evitar o pior caso do quicksort na prática?

Resposta. Não usar pivô fixo no primeiro/último elemento: escolher o **elemento do meio**, um **pivô aleatório** (`random.choice`) ou a **mediana de três**. Com pivô aleatório, a chance de cair sempre no pior particionamento é desprezível, e o desempenho **esperado** é $O(n \log n)$ — foi o “caso médio” do experimento da aula 06 (lista de 1.000 embaralhada). Também é útil aumentar o limite de recursão ou usar uma versão iterativa para listas muito grandes.

6.13 Quicksort × merge sort: os dois são $O(n \log n)$ no caso médio, mas o merge sort é $O(n \log n)$ **também** no pior caso. Então o merge sort é sempre a melhor escolha?

Resposta. Não necessariamente. Big O esconde a **constante**: o quicksort faz menos trabalho por nível (particiona no lugar, não precisa intercalar) e usa menos memória, então na prática costuma ser **mais rápido** — por isso a qsort da biblioteca padrão do C. Com pivô aleatório, o pior caso $O(n^2)$ do quicksort é raríssimo, e o caso médio é o que importa. O merge sort vale quando é preciso **garantir** o pior caso (sistemas de tempo real) ou estabilidade, e custa $O(n)$ de memória extra.

6.14 Simule o merge sort em [38, 27, 43, 3, 9, 82, 10].

Resposta. Dividir: [38, 27, 43] e [3, 9, 82, 10] → [38] [27, 43] e [3, 9] [82, 10] → ... Combinar: [27] + [43] → [27, 43]; [38] + [27, 43] → [27, 38, 43]; [3] + [9] → [3, 9]; [82] + [10] → [10, 82]; [3, 9] + [10, 82] → [3, 9, 10, 82]; final: [27, 38, 43] + [3, 9, 10, 82] → **[3, 9, 10, 27, 38, 43, 82]**. Altura $\approx \log_2 7 \approx 3$ níveis; cada nível intercala n elementos → $O(n \log n)$.

6.15 Qual é o caso base do quicksort e por que ele é suficiente para ordenar um array de qualquer tamanho?

Resposta. Arrays com **0 ou 1 elemento** já estão ordenados — basta devolvê-los. Indução: se você sabe ordenar arrays de 0 a $k-1$ elementos, então sabe ordenar um de k : escolha um pivô, particione (os dois subarrays têm entre 0 e $k-1$ elementos) e chame o quicksort neles. Como vale para 0 e 1, vale para 2, depois 3, e assim por diante — **qualquer pivô funciona**, só muda a eficiência.

Tema 7 — Questões discursivas e cenários (estilo prova)

7.1 Por que, ao comparar dois algoritmos, é importante medir **passos** e **tempo**, e não apenas um deles?

Resposta. Passos (comparações, chamadas) são propriedade do **algoritmo**: dão o mesmo número em qualquer máquina e provam a classe Big O. Tempo é propriedade da **execução**: sofre influência de CPU, linguagem, cache, sistema operacional — mas é o que o usuário sente e captura as constantes que Big O ignora. Quando as duas evidências concordam, a conclusão é robusta; quando divergem (ex.: 50.000× menos passos mas 2.300× menos tempo), aprende-se algo sobre as constantes.

7.2 Por que usar **mediana** ou **mínimo** em vez de **média** ao medir tempo de execução? E por que repetir a medição?

Resposta. Uma única execução de uma busca binária dura $\approx 0,002$ ms — dentro do ruído do relógio. Repetir muitas vezes torna a medição maior que o ruído. A **média** é puxada por picos causados pelo sistema operacional (outros processos, coletor de lixo); a **mediana** e o **mínimo** ignoram esses picos e refletem o custo típico ou o custo “limpo” do algoritmo.

7.3 Por que se deve **validar a corretude** dos algoritmos antes de fazer o benchmark? Como você faria isso?

Resposta. Um algoritmo rápido e **errado** não serve como base de comparação: o resultado do benchmark seria inútil. Validação: gerar muitos alvos/listas aleatórias, comparar a saída dos algoritmos entre si e com uma referência (`sorted()`, `lista.index()`, o Fibonacci iterativo), usando `assert`; para a busca binária, conferir também que os passos nunca passam de $\lfloor \log_2 n \rfloor + 1$.

7.4 Para que serve `random.seed(42)` em um experimento?

Resposta. Reprodutibilidade. Fixa a sequência de números “aleatórios”, então cada execução sorteia os mesmos alvos e as mesmas listas. Sem isso, cada rodada mede um teste diferente e comparar execuções não faz sentido.

7.5 Cenário: um sistema de login tem 10 milhões de usuários em uma lista **ordenada** por nome. As buscas são frequentes e as inserções, raras. Que estrutura e algoritmo você usaria? Quantos passos, no pior caso, tem cada busca?

Resposta. Array ordenado + busca binária. $\log_2 10.000.000 \approx 23,3 \rightarrow$ **24 passos** no pior caso, contra 10 milhões na busca linear. Inserções raras toleram o custo $O(n)$ de manter o array ordenado. (É o raciocínio do exercício 2.3 do livro.)

7.6 Cenário: você precisa ordenar **1 milhão** de registros em ordem aleatória. Um colega propõe bubble sort porque “é simples”. Argumente com números.

Resposta. Bubble sort: $\approx n^2/2 = 5 \times 10^{11}$ comparações (e trocas na mesma ordem). Quicksort/merge sort: $\approx n \cdot \log_2 n \approx 2 \times 10^7$. Diferença de $\approx$ **25.000x**. Se cada comparação levasse 10 ns, o bubble sort demoraria $\approx 1,4$ h e o quicksort $\approx 0,2$ s. Bubble sort só se justifica para listas pequenas ou quase ordenadas.

7.7 Cenário: uma lista de 100 registros muito grandes (cada troca copia megabytes) chega **quase** ordenada, com 2 ou 3 elementos fora do lugar. Bubble sort ou selection sort?

Resposta. Bubble sort. Quase ordenada $\rightarrow$ poucas passagens e poucas trocas (só as necessárias para os 2 ou 3 elementos), tendendo a $O(n)$. O selection sort faria sempre 4.950 comparações; suas trocas seriam poucas, mas o bubble também trocaria pouco nesse caso. Se a lista fosse **aleatória** e a troca cara, aí o selection sort (≤ 99 trocas) venceria o bubble (≈ 2.500 trocas).

7.8 Cenário: um programa precisa calcular $F(n)$ para **muitos** valores de n , repetidamente, ao longo da execução. Qual versão do Fibonacci você usaria e por quê?

Resposta. A memoizada (@lru_cache ou dicionário **persistente** entre chamadas). Uma vez calculado, cada $F(k)$ fica guardado e as consultas repetidas custam $O(1)$ — o cenário em que o cache mais brilha. A iterativa recalcularia do zero a cada chamada ($O(n)$ cada), e a recursiva pura é inviável ($O(2^n)$). Se n puder ser muito grande (milhares), preencher o cache **de baixo para cima** evita o estouro de pilha.

7.9 Um algoritmo mais rápido “no papel” (melhor Big O) pode ser mais lento na prática? Dê dois exemplos.

Resposta. Sim. (1) **Quicksort $\times$ merge sort:** mesmo Big O médio, mas o quicksort tem constante menor e vence na prática. (2) **Busca linear $\times$ ordenar + busca binária** para poucas consultas: $O(n)$ direto vence $O(n \log n) + O(\log n)$. (3) Para n pequeno, um $O(n^2)$ simples pode vencer um $O(n \log n)$ cheio de chamadas recursivas. Big O descreve o comportamento **assintótico** (n grande); para n pequeno, constantes e custos fixos dominam.

7.10 Explique a diferença entre a **altura da pilha** e o **número total de chamadas** em um algoritmo recursivo. Use o quicksort e o Fibonacci como exemplos.

Resposta. Altura da pilha = quantas chamadas estão empilhadas **ao mesmo tempo** (memória; risco de estouro). **Total de chamadas** = quantas vezes a função foi executada (tempo). Quicksort: altura $\log n$ (melhor) ou n (pior); total de chamadas sempre $O(n)$ — entre n e $2n - 1$ (cada chamada com 2 ou mais elementos gera duas novas; no exemplo 6.7 foram 7 chamadas para $n = 5$, no pior caso 6.9 foram 9). O que muda entre melhor e pior caso não é o número de chamadas, e sim a altura da pilha e o trabalho por nível. Fibonacci recursivo puro: altura n , mas total $2 \cdot F(n+1) - 1 \approx 1,618^n$. Fibonacci memoizado: altura n , total $2n - 1$.

7.11 Complete a frase e justifique: “Quando a entrada dobra, um algoritmo $O(n^2)$ faz ____ vezes mais trabalho; um $O(\log n)$ faz apenas ____ a mais; um $O(2^n)$ tem seu trabalho ____.”

Resposta. 4 vezes mais ($2^2 = 4$); **um passo** a mais ($\log_2 2n = \log_2 n + 1$); **elevado ao quadrado** ($2^{2n} = (2^n)^2$).

PARTE 4 — Simulado rápido (gabarito no fim da parte)

- S1.** A busca binária em uma lista de 1.024 elementos faz, no máximo: (a) 1.024 passos (b) 512 passos (c) 10 passos (d) 32 passos.
- S2.** Qual algoritmo faz **sempre** o mesmo número de comparações, independentemente da ordem inicial? (a) bubble sort com parada (b) selection sort (c) quicksort (d) busca linear.
- S3.** O pior caso do quicksort com pivô no primeiro elemento ocorre quando: (a) a lista é aleatória (b) a lista já está ordenada (c) a lista tem elementos repetidos (d) a lista tem tamanho ímpar.
- S4.** Verdadeiro ou falso: “ $O(n/26)$ é uma classe diferente de $O(n)$.”
- S5.** O Fibonacci recursivo puro tem complexidade de tempo e de espaço, respectivamente: (a) $O(n)$ e $O(n)$ (b) $O(2^n)$ e $O(2^n)$ (c) $O(2^n)$ e $O(n)$ (d) $O(n)$ e $O(1)$.
- S6.** Quantas comparações o bubble sort faz em [1, 2, 3, 4, 5, 6] já ordenado, com parada quando não há troca? (a) 15 (b) 6 (c) 5 (d) 0.
- S7.** Qual das etapas **não** faz parte do “dividir para conquistar”? (a) dividir (b) conquistar (c) combinar (d) ordenar.
- S8.** maior_quadrado(100, 75) devolve: (a) 75 (b) 25 (c) 50 (d) 5.
- S9.** Verdadeiro ou falso: “A busca binária funciona em qualquer lista, ordenada ou não, desde que se saiba o tamanho.”
- S10.** Um algoritmo faz 400 operações para $n = 20$ e 1.600 para $n = 40$. Sua classe provável é: (a) $O(n)$ (b) $O(n \log n)$ (c) $O(n^2)$ (d) $O(2^n)$.
- S11.** Sem caso base, uma função recursiva em Python gera: (a) resultado zero (b) laço infinito silencioso (c) RecursionError (estouro de pilha) (d) resposta correta mais lenta.
- S12.** Quantas chamadas faz o Fibonacci memoizado para $n = 50$? (a) 50 (b) 99 (c) 2^{50} (d) 2.500.
- S13.** Ordenar uma lista com merge sort e com quicksort (pivô aleatório) tem, no caso médio, Big O: (a) iguais, $O(n \log n)$ (b) $O(n^2)$ e $O(n \log n)$ (c) $O(n \log n)$ e $O(n^2)$ (d) $O(n)$ e $O(n \log n)$.
- S14.** Em um experimento de tempo, a medida menos sensível a picos do sistema operacional é: (a) média (b) soma (c) mediana ou mínimo (d) primeira execução.
- S15.** O selection sort em [9, 7, 5, 3, 1] ($n = 5$) faz quantas comparações e quantas trocas? (a) 10 e 10 (b) 10 e 2 (c) 4 e 0 (d) 5 e 5.
- Gabarito:** S1 (c) $2^{10} = 1.024$ · S2 (b) · S3 (b) · S4 Falso — constantes são ignoradas · S5 (c) · S6 (c) $n-1 = 5$ comparações, 1 passagem · S7 (d) · S8 (b) $100 \bmod 75 = 25$; $75 \bmod 25 = 0$ · S9 Falso · S10 (c) dobrou n , quadruplicou o trabalho · S11 (c) · S12 (b) $2n - 1$ · S13 (a) · S14 (c) · S15 (b) 10 comparações, 2 trocas ($9 \leftrightarrow 1$, $7 \leftrightarrow 3$).
-

PARTE 5 — Tabelas numéricas para consulta

5.1 Potências de 2 (passos da busca binária = k para n = 2^k)

k	2 ^k	k	2 ^k	k	2 ^k
1	2	8	256	15	32.768
2	4	9	512	16	65.536
3	8	10	1.024	17	131.072
4	16	11	2.048	18	262.144
5	32	12	4.096	19	524.288
6	64	13	8.192	20	1.048.576
7	128	14	16.384	30	≈ 1,07 bilhão
				32	≈ 4,29 bilhões

Regra: passos da busca binária para n itens = menor k tal que 2^k ≥ n. Ex.: n = 100 → 7 (2⁷ = 128); n = 1.000 → 10; n = 1.000.000 → 20; n = 10.000.000 → 24.

5.2 n(n-1)/2 — comparações do selection sort e do bubble sort (pior caso)

n	n(n-1)/2	n	n(n-1)/2
5	10	50	1.225
6	15	100	4.950
8	28	500	124.750
10	45	1.000	499.500
20	190	1.000.000	≈ 5 × 10 ¹¹

5.3 Fibonacci e número de chamadas

n	F(n)	Chamadas recursivo puro 2·F(n+1) – 1	Chamadas memoizado 2n – 1
0	0	1	—
1	1	1	1
2	1	3	3
3	2	5	5
4	3	9	7
5	5	15	9
6	8	25	11
7	13	41	13
8	21	67	15
9	34	109	17
10	55	177	19
15	610	1.973	29

n	F(n)	Chamadas recursivo puro 2·F(n+1) – 1	Chamadas memoizado 2n – 1
20	6.765	21.891	39
25	75.025	242.785	49
30	832.040	2.692.537	59

Razão áurea $\phi = (1 + \sqrt{5})/2 \approx 1,618$: a cada +1 em n, as chamadas do recursivo puro multiplicam por $\approx 1,618$.

5.4 Fatoriais (rotas do caixeiro-viajante)

n	n!	n	n!
3	6	8	40.320
4	24	9	362.880
5	120	10	3.628.800
6	720	12	479.001.600
7	5.040	16	$\approx 2,09 \times 10^{13}$

5.5 Logaritmos úteis (base 2)

$\log_2 10 \approx 3,32 \cdot \log_2 100 \approx 6,64 \cdot \log_2 1.000 \approx 9,97 \cdot \log_2 10.000 \approx 13,3 \cdot \log_2 100.000 \approx 16,6 \cdot \log_2 1.000.000 \approx 19,9 \cdot \log_2 10^7 \approx 23,3 \cdot \log_2 10^9 \approx 29,9$.

Fontes: material das aulas 03–06 (Prof. Humberto Zanetti, Fatec Jundiaí); atividades 01, 02 e 03 desta disciplina; BHARGAVA, Aditya Y. **Entendendo Algoritmos**. Novatec, caps. 1–4.